Subqueries

1. DATABASE

1. To send a part of data to any other user then we cant send whole data to user which kills the storage and may cause the Performance Issues,Redundancy , Security Issues, Hard to maintain whole data, complexity increases
2. so we can use
    a. SUBQUERY
    b. CTE (common table expression)
    c. VIEWS
    d. TEMP TABLES
    e. CTAS (create table as select)

2. DATABASE Architecture

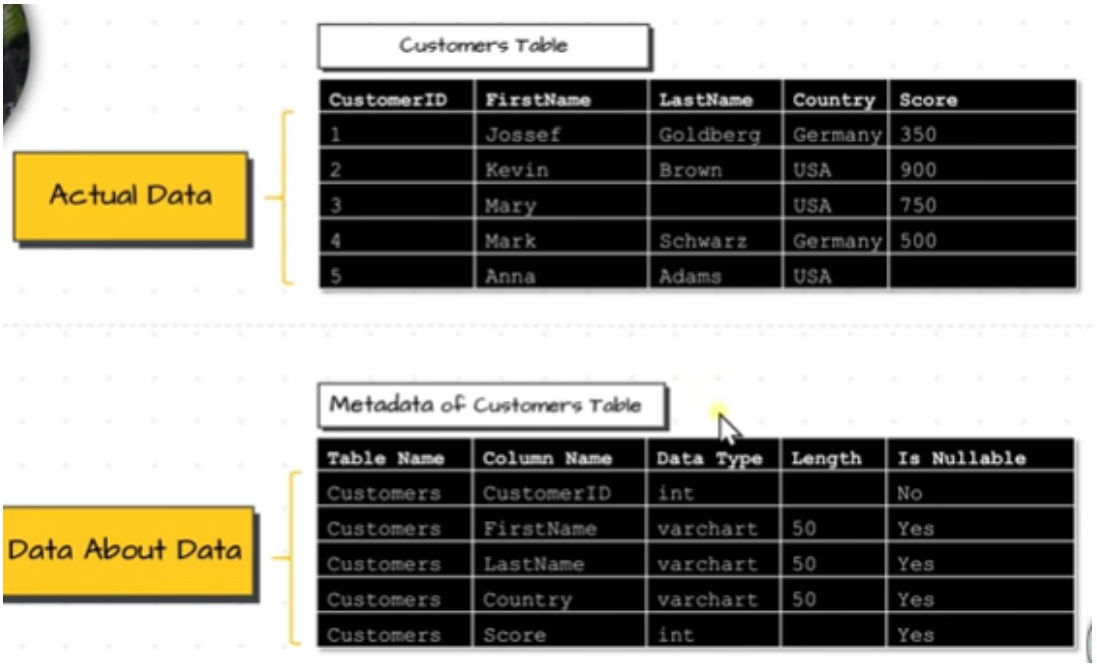
1. To retrieve data, create data, or to do any application on data in the database → client/user sends the query to database .
2. But he cannot send query directly to database , to manage requests and for security purpose DATABASE ENGINE helps to manage our requests
3. DATABASE ENGINE → It is the brain of the database, executing multiple operations such as storing.retrieving,and managing data within the database
    a. Capacity → can hold large data
4. DISK STORAGE → Long-term memory , where data is stored permanent
    a. Speed → slow to read and write
5. CACHE MEMORY → Fast short-term memory , where data is stored temporarily
    a. Speed → extremely fast to read and to write
    b. Capacity → small
6. DISK STORAGE → has three parts

a. USER → USER DATA STORAGE → Its the main cotent of the database , this is where the actual data that users about is stored

b. SYSTEM CATALOG → Database internal storage for its own information ,

   i. It means it stores the schema of the data to be stored

   ii. They call it as METADATA → Data about Data

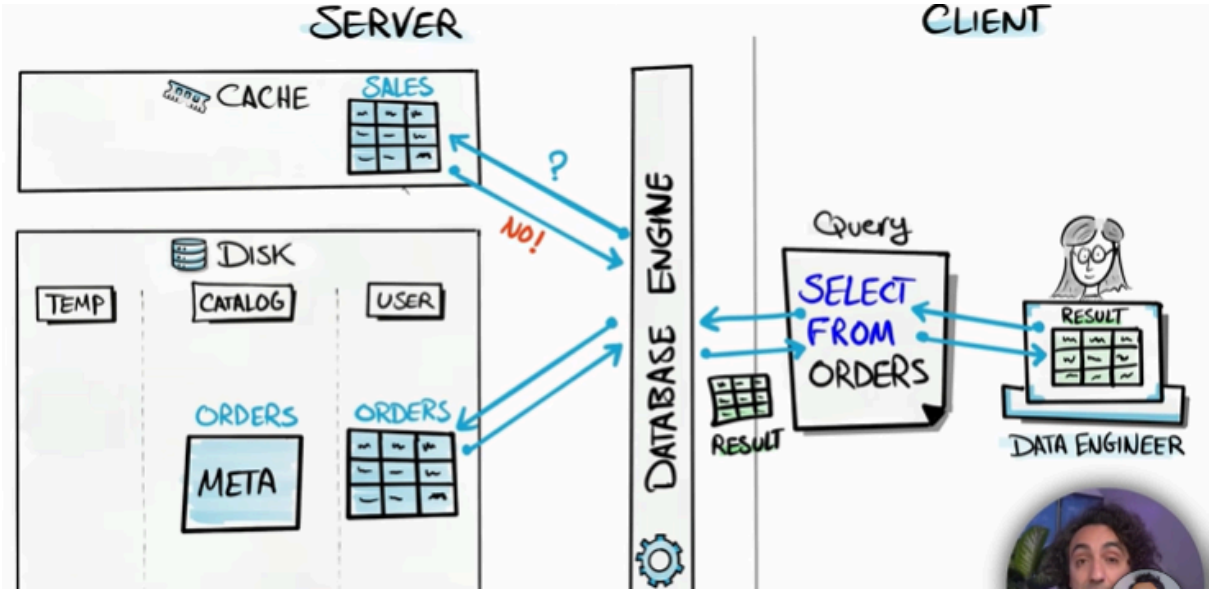


c. INFORMATION SCHEMA → a system-defined schems with build in views that provide info about the database like tables and columns

7. TEMP DATA STORAGE → Temporary space used by the database for short-term tasks, like processing queries or sorting data

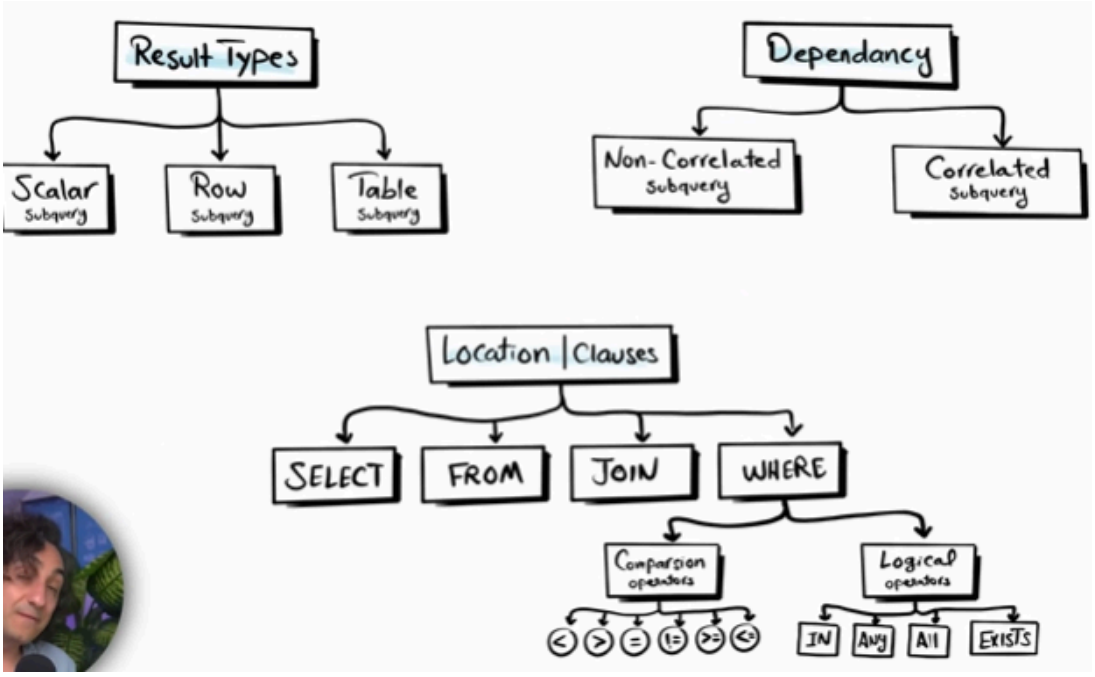
   a. Once these tasks are done, the strage is cleared

8. CACHE → most of the Queires we work frequently will be here



3. SUBQUERY

1. A query inside another query
2. Topics



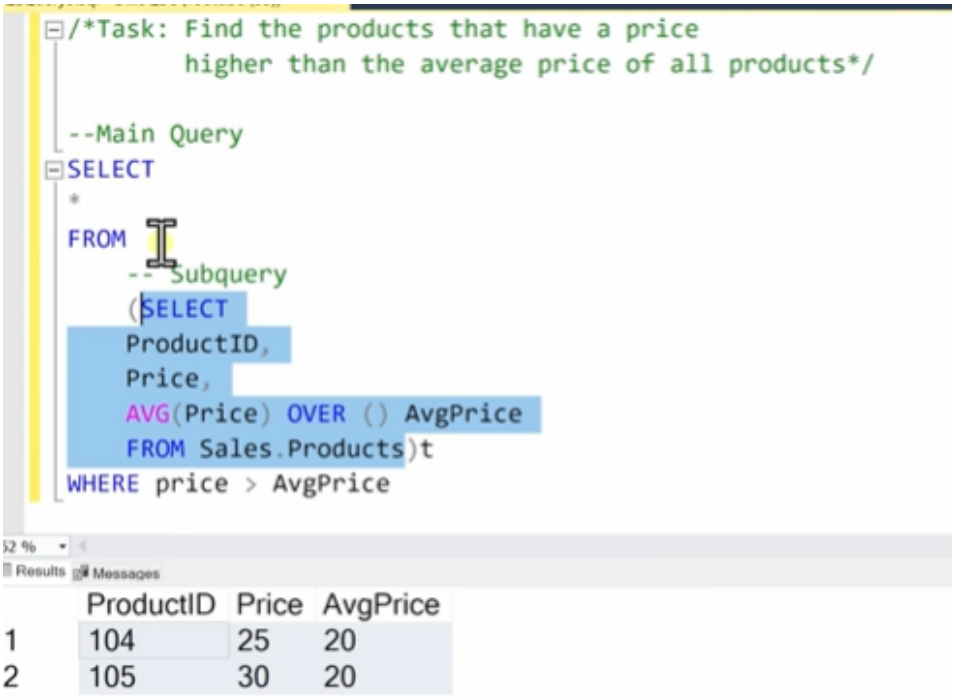
a. Result-types

1. SCALAR subquery → Means that sub-query should return only single value (single row,single column)
    a. Example → SELECT SUM(rate) FROM products

2. ROW subquery → Multiple Rows and single column
3. TABLE subquery → Multiple Rows and Multiple Columns

b. Location/Clauses

1. FROM → Used as temporary table for the main query
    a. Sub-query should be kept in position FROM
    b. So SELECT will takes the columns from that TABLE generated from the SUB-QUERY
    c. so that SUB_QUERY must be ROW/TABLE subquery
    d. ALIAS for the sub-query must be required

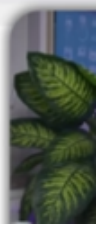


    a. Here the subquery returns the single column data
    b. First the subquery will be executed , then Main query
2. SELECT → to select whole data from the subquery
    a. So that SUB_QUERY must be return only one value → Means SCALAR SUB_QUERY only allowed
    b. ALIAS required

```sql
-- Show the product IDs, product names, prices, and the total number of orders

-- Main Query
SELECT
    ProductID,
    Product,
    Price,
    -- Subquery
    (SELECT COUNT(*) FROM Sales.Orders) AS TotalOrders
FROM Sales.Products
```

	ProductID	Product	Price	TotalOrders
1	101	Bottle	10	10
2	102	Tire	15	10
3	103	Socks	20	10
4	104	Caps	25	10
5	105	Gloves	30	10



c. here the sub-query returns only one value

3. JOIN → Used to prepare the data(filtering or aggregation) before joining it with other tables
    a. when we join two tables → the subquery will be one of the table

```sql
-- Show all customer details and find the total orders of each customer

--Main Query
SELECT
c.*,
o.TotalOrders
FROM Sales.Customers c
LEFT JOIN (
    SELECT
    CustomerID,
    COUNT(*) TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID) o
ON c.CustomerID = o.CustomerID
```

	CustomerID	FirstName	LastName	Country	Score	TotalOrders
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	NULL

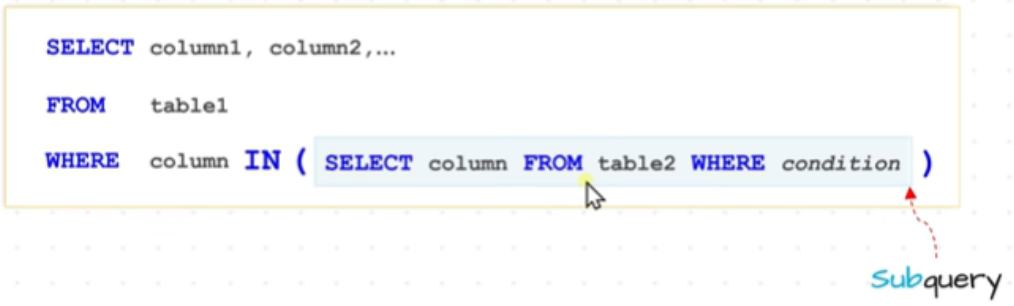
4. WHERE → Used for complex filtering logic and makes query more flexible and dynamic

a. COMPARISON OPERATORS → Used to filte data by comparing two values

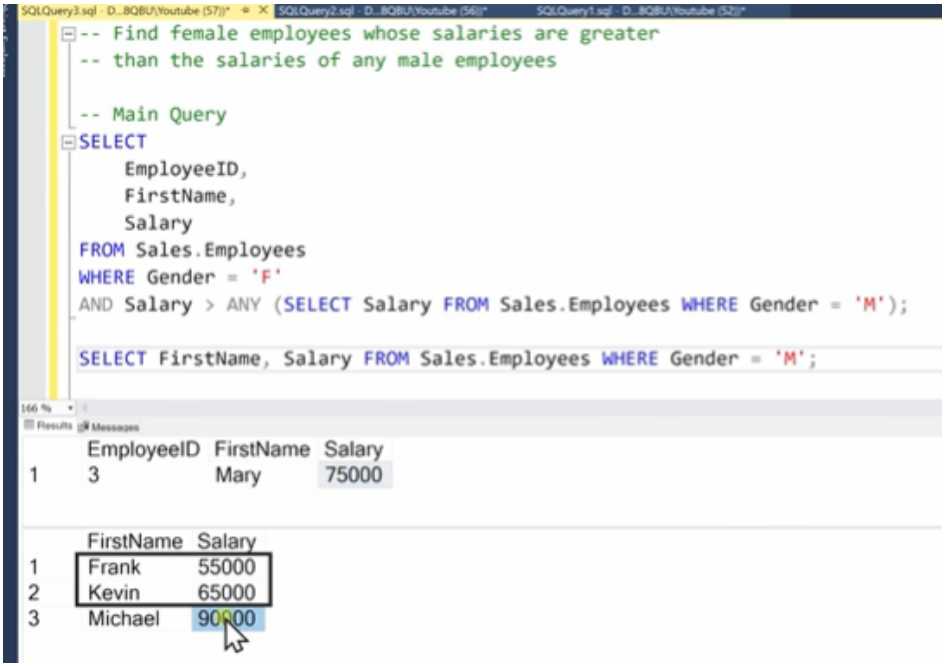


the sub-query must be return only single value

b. LOGICAL OPERATORS
    a. IN OPERATOR → checks whether a value matches any value from a list
        a. So for IN operator , the subquery must return a single column or single value

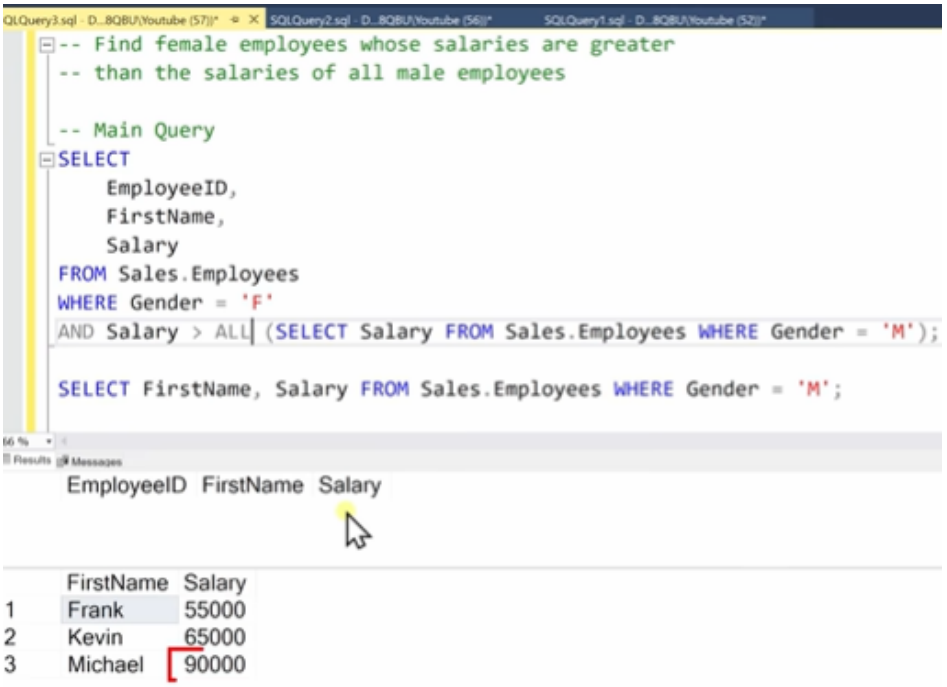


    b. ANY OPERATOR → checks if a value matches ANY value within a list
        a. Used to check if a value is true for AT LEAST one of the values in a list



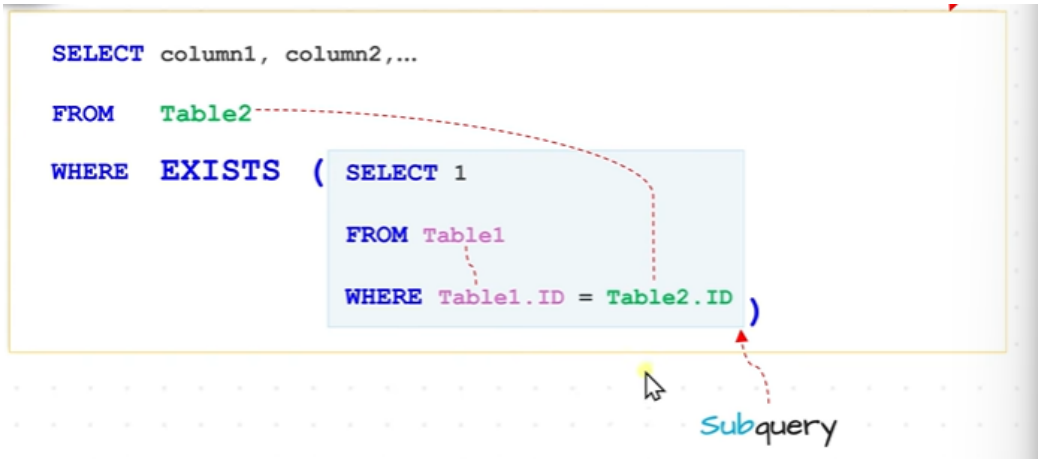
Here onlt MARY has amount graeter than atleast one value from males salary

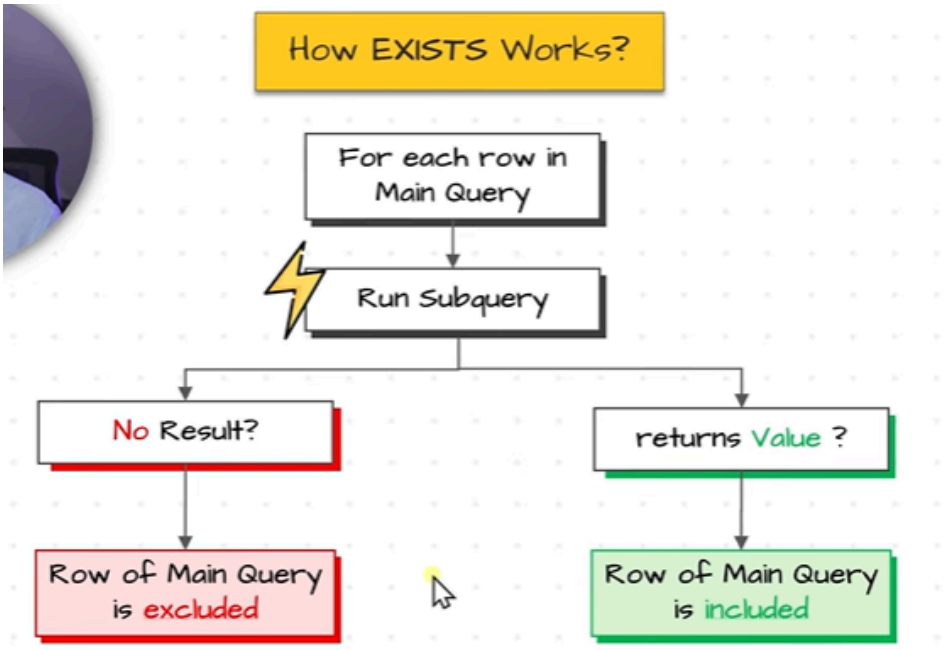
c. ALL OPERATOR → checks if a value matches ALL value within a list



d. EXISTS → Check if a subquery returns any rows

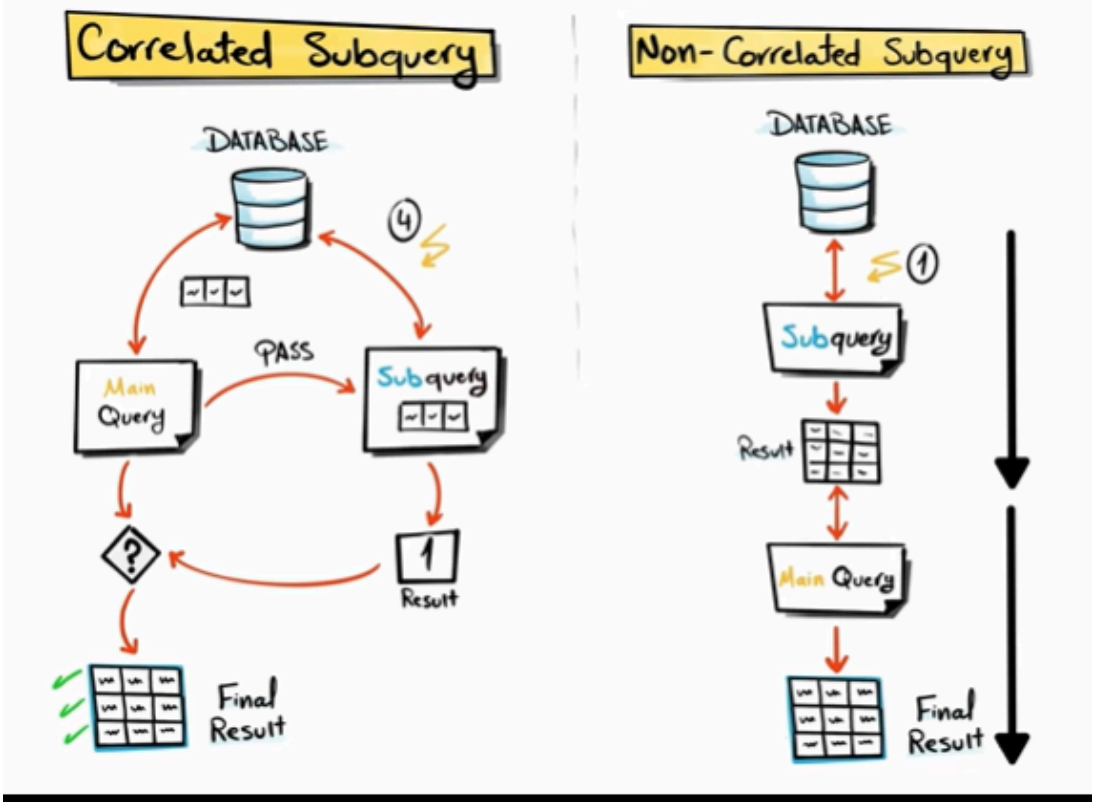
   a. Used in Correlated subquery





c. Dependency

1. NON-CORRELATED → A Subquery that can run independently from the Main Query
    a. These are what we seen upto now
2. CORRELATED SUBQUERY → A Subquery that relays on values from the Main Query
    a. Checks every row of Main Query with the every row of the Sub-query




```
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = c.CustomerID) TotalSales
FROM Sales.Customers c
```

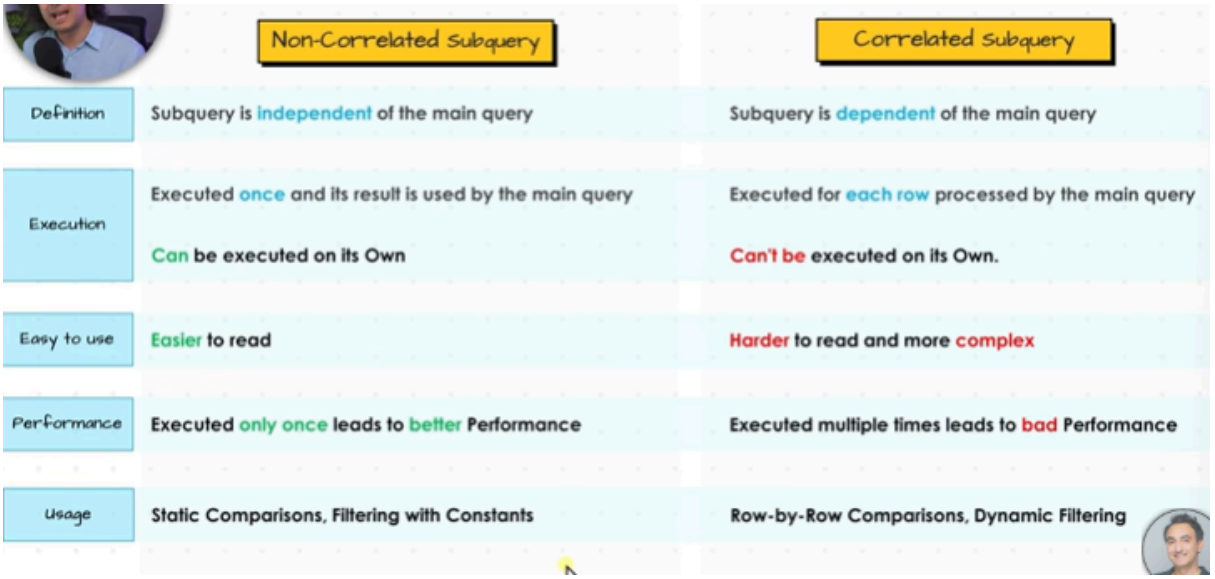
	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	0



b. Here in subquery if we see that it depends on the Main-query also

c. Here for every row of Orders it checks condition to every row of Customers

d. Correlated vs Non-Correlated Sub-query


Non-Correlated Subquery
Correlated Subquery
Definition
Subquery is independent of the main query
Subquery is dependent of the main query
Execution
Executed once and its result is used by the main query
Can be executed on its Own
Executed for each row processed by the main query
Can't be executed on its Own.
Easy to use
Easier to read
Harder to read and more complex
Performance
Executed only once leads to better Performance
Executed multiple times leads to bad Performance
Usage
Static Comparisons, Filtering with Constants
Row-by-Row Comparisons, Dynamic Filtering